

HPC LAB 04 – U23AI060

Problem Statement 1 : Parallel Array Summation via Task Decomposition

```
#include <stdio.h>
#include <omp.h>

#define SIZE 600
#define STEP_SIZE 100

int main() {
    int arr[SIZE];
    int sum = 0;

    for(int i = 0; i < SIZE; i++) {
        arr[i] = i + 1;
    }

    printf("\n==== Parallel Array Summation using OpenMP Tasks =====\n");

    #pragma omp parallel shared(sum, arr)
    {
        #pragma omp single
        {
            printf("Task generator thread: %d\n", omp_get_thread_num());

            for(int i = 0; i < SIZE; i += STEP_SIZE) {
                int start = i;
                int end = (i + STEP_SIZE < SIZE) ? i + STEP_SIZE : SIZE;

                printf("Thread %d generating task for range [%d - %d]\n",
                    omp_get_thread_num(), start, end - 1);

                #pragma omp task firstprivate(start, end) shared(sum, arr)
                {
                    int psum = 0;

                    printf("  -> Task executing on thread %d for range [%d - %d]\n",
                        omp_get_thread_num(), start, end - 1);

                    for(int j = start; j < end; j++) {
                        psum += arr[j];
                    }

                    #pragma omp critical
                    {
                        sum += psum;
                        printf("    -> Thread %d added partial sum %d, global sum now = %d\n",
                            omp_get_thread_num(), psum, sum);
                    }
                }
            }
        }
    }

    printf("\nFinal Global Sum = %d\n", sum);

    return 0;
}
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/04_lab_$ gcc -fopenmp 01_q.c -o p1
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/04_lab_$ ./p1

==== Parallel Array Summation using OpenMP Tasks =====
Task generator thread: 6

Thread 6 generating task for range [0 - 99]
Thread 6 generating task for range [100 - 199]
  -> Task executing on thread 3 for range [0 - 99]
  -> Thread 3 added partial sum 5050, global sum now = 5050
  -> Task executing on thread 13 for range [100 - 199]
  -> Thread 13 added partial sum 15050, global sum now = 20100
Thread 6 generating task for range [200 - 299]
Thread 6 generating task for range [300 - 399]
Thread 6 generating task for range [400 - 499]
  -> Task executing on thread 9 for range [300 - 399]
  -> Thread 9 added partial sum 35050, global sum now = 55150
  -> Task executing on thread 2 for range [200 - 299]
  -> Thread 2 added partial sum 25050, global sum now = 80200
  -> Task executing on thread 9 for range [400 - 499]
  -> Thread 9 added partial sum 45050, global sum now = 125250
Thread 6 generating task for range [500 - 599]
  -> Task executing on thread 9 for range [500 - 599]
  -> Thread 9 added partial sum 55050, global sum now = 180300

Final Global Sum = 180300
```

Analysis:

=====Analysis of Output=====

=Parallel execution observed → Multiple threads create and execute tasks.

=Dynamic scheduling → Task creator and executor threads may be different.

=Non-sequential execution → Array ranges may execute in random order.

=Race condition prevented → #pragma omp critical ensures correct global sum.

=Concurrency achieved → Multiple tasks run simultaneously, improving efficiency.

=Conclusion: The program successfully demonstrates task-based parallelism with proper synchronization and correct summation.

Problem Statement 2 : Analysing the variable access using different clause

```
04_lab_ > C 02_q_c
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int x = 10;
6
7      printf("Initial value of x in main: %d\n\n", x);
8
9      #pragma omp parallel
10     {
11         #pragma omp single
12         {
13             #pragma omp task shared(x)
14             {
15                 x = x + 5;
16                 printf("SHARED Task -> x = %d, Thread = %d\n", x, omp_get_thread_num());
17             }
18
19             #pragma omp task private(x)
20             {
21                 x = 100;
22                 printf("PRIVATE Task -> x = %d, Thread = %d\n", x, omp_get_thread_num());
23             }
24
25             /
26             #pragma omp task firstprivate(x)
27             {
28                 x = x + 20;
29                 printf("FIRSTPRIVATE Task -> x = %d, Thread = %d\n", x, omp_get_thread_num());
30             }
31
32             #pragma omp taskwait
33         }
34
35         printf("\nFinal value of x in main: %d\n", x);
36
37         return 0;
38     }
39
40 }
41
42
43
44 Analysis :
45
46 Shared variable: The shared task modifies the same memory location, so changes affect the final value of x.
47
48 Private variable: The private task creates a new independent variable, so its changes do not affect the main variable.
49
50 Firstprivate variable: The firstprivate task gets a copy of the parent value at task creation and modifies its local copy only, without changing the main variable.
51
52 Conclusion: The output shows how OpenMP handles different variable scopes in tasks and ensures controlled data sharing and isolation.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/04_lab_$ gcc -fopenmp 02_q_c -o p2
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/04_lab_$ ./p2
Initial value of x in main: 10

SHARED Task -> x = 15, Thread = 11
PRIVATE Task -> x = 100, Thread = 13
FIRSTPRIVATE Task -> x = 35, Thread = 15

Final value of x in main: 15
```

Problem Statement 3 : Analyzing Task Creation and Execution in OpenMP

```
04_lab_ > C 03_q_c
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5
6      omp_set_num_threads(4);
7
8      #pragma omp parallel
9      {
10         #pragma omp single
11         {
12             printf("Producer Thread: %d\n\n", omp_get_thread_num());
13
14             for(int i = 0; i < 8; i++) {
15
16                 int creator_thread = omp_get_thread_num();
17
18                 #pragma omp task firstprivate(i, creator_thread)
19                 {
20                     int executor_thread = omp_get_thread_num();
21
22                     printf("Hello World | Loop index = %d | Created by = %d | Executed by = %d\n",
23                             i, creator_thread, executor_thread);
24                 }
25             }
26
27             #pragma omp taskwait
28         }
29     }
30
31     return 0;
32 }
33
34 // Analysis:
35 - One thread (producer) creates tasks.
36
37 - Tasks are executed by any available worker thread.
38
39 - Shows difference between task generation vs execution.
40
41 - Task creation != task execution.
42
43 - OpenMP runtime schedules tasks dynamically.
44
45 - Load balancing happens automatically.
46
47 Thread Behavior:
48
49 Single thread produces tasks.
50
51 Multiple threads consume tasks.
52
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/04_lab_$ ./p3
Producer Thread: 2

Hello World | Loop index = 0 | Created by = 2 | Executed by = 0
Hello World | Loop index = 3 | Created by = 2 | Executed by = 0
Hello World | Loop index = 4 | Created by = 2 | Executed by = 0
Hello World | Loop index = 5 | Created by = 2 | Executed by = 0
Hello World | Loop index = 6 | Created by = 2 | Executed by = 0
Hello World | Loop index = 1 | Created by = 2 | Executed by = 1
Hello World | Loop index = 2 | Created by = 2 | Executed by = 3
Hello World | Loop index = 7 | Created by = 2 | Executed by = 2
```

Problem Statement 4:

Objective: Implement a parallel program that traverses a singly linked list and processes each node's data using OpenMP Tasks.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

void processNode(Node* node) {
    printf("Processing node value = %d by Thread = %d\n",
        node->data, omp_get_thread_num());
}

int main() {

    Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);
    head->next->next->next = createNode(40);

    omp_set_num_threads(4);

    #pragma omp parallel
    {

        #pragma omp single
        {
            Node* temp = head;

            while(temp != NULL) {

                #pragma omp task firstprivate(temp)
                {
                    processNode(temp);
                }

                temp = temp->next;
            }

            #pragma omp taskwait
        }
    }

    return 0;
}
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/04_lab_$ gcc -fopenmp 04_q_.c -o p4
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/04_lab_$ ./p4
Processing node value = 10 by Thread = 0
Processing node value = 20 by Thread = 3
Processing node value = 30 by Thread = 1
Processing node value = 40 by Thread = 2
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/04_lab_$
```

Analysis:

- Single thread traverses linked list → producer.
- Creates tasks for each node.
- Worker threads process nodes → consumers.
- `firstprivate(temp)` ensures correct node pointer.

Without `firstprivate`:

- All tasks may access same pointer.
- Wrong node processed.
- Race condition.

Design Pattern Used:

- Producer-Consumer model.